

Chapter 3: Searching, Sorting, and Complexity

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives

- Determine the rate of growth of the work of an algorithm in terms of its problem size
- Use big-O notation to describe the running time and memory usage of an algorithm
- Recognize the common rates of growth of work, or complexity classes – constant, logarithmic, linear, quadratic, and exponential
- Convert an algorithm to a faster version that reduces its complexity by an order of magnitude
- Describe how the sequential search and binary search algorithms work
- Describe how the selection sort and quicksort algorithms work

Measuring the Efficiency of Algorithms

- When choosing algorithms,
 - You often have to settle for a space/time trade-off
- An algorithm can be designed to gain faster run times
 - At the cost of using extra space (memory) or the other way around
- Space/time trade-off is more relevant for miniature devices

Measuring the Run Time of an Algorithm (1 of 6)

- Use the computer's clock to obtain an actual run time:
 - Process is called benchmarking or profiling
- Starts by determining the time for several different data sets of the same size and then calculates the average time
- Next, similar data are gathered for larger and larger data sets:
 - After several tests, enough data are available to predict how the algorithm will behave for a data set of any size
- Code for a tester program is found on the following slide

Measuring the Run Time of an Algorithm (2 of 6)

```
"""
File: timing1.py
Prints the running times for problem sizes that double,
using a single loop.
"""
```

```
import time

problemSize = 10000000
print("%12s%16s" % ("Problem Size", "Seconds"))
for count in range(5):
    start = time.time()
    # The start of the algorithm
    work = 1
    for x in range(problemSize):
        work += 1
        work -= 1
    # The end of the algorithm
    elapsed = time.time() - start
    print("%12d%16.3f" % (problemSize, elapsed))
    problemSize *= 2
```

Measuring the Run Time of an Algorithm (3 of 6)

Figure 3-1: The output of the tester program

Problem Size	Seconds
10000000	3.8
20000000	7.591
40000000	15.352
80000000	30.697
160000000	61.631

Measuring the Run Time of an Algorithm (4 of 6)

- As another example, consider the following change in the tester's program's algorithm:

```
for j in range(problemSize):  
    for k in range(problemSize):  
        work += 1  
        work -= 1
```

- The extended assignments have been moved into a nested loop:
 - Loop iterates through the size of the problem within another loop that also iterates through the size of the problem
 - The results are shown in Figure 3.2 on the following slide.

Measuring the Run Time of an Algorithm (5 of 6)

Figure 3-2: The output of the second tester program

Problem Size	Seconds
1000	0.387
2000	1.581
4000	6.463
8000	25.702
16000	102.666

Measuring the Run Time of an Algorithm (6 of 6)

- This method permits accurate predictions of running times of many algorithms with only two problems:
 - Different hardware platforms have different processing speeds, so the running times of an algorithm differ from machine to machine
 - It is impractical to determine the running time for some algorithms with very large data sets

Counting Instructions (1 of 5)

- Another technique to estimate efficiency of an algorithm
 - Count the instructions executed with different problem sizes
- When analyzing an algorithm this way, you distinguish between two classes of instructions:
 - Instructions that execute the same number of times regardless of the problem size
 - Instructions whose execution count varies with the problem size
- For now, ignore the instructions in the first class:
 - They do not figure significantly in this kind of analysis
- The instructions in the second class normally are found in loops or recursive functions:
 - In the case of loops, zero in on instructions performed in any nested loops

Counting Instructions (2 of 5)

```
"""
File: counting.py
Prints the number of iterations for problem sizes
that double, using a nested loop.
"""

problemSize = 1000
print("%12s%15s" % ("Problem Size", "Iterations"))
for count in range(5):
    number = 0
    # The start of the algorithm
    work = 1
    for j in range(problemSize):
        for k in range(problemSize):
            number += 1
            work += 1
            work -= 1
    # The end of the algorithm
    print("%12d%15d" % (problemSize, number))
    problemSize *= 2
```

Counting Instructions (3 of 5)

Figure 3-3: The output of a tester program that counts iterations

Problem Size	Iterations
1000	1000000
2000	4000000
4000	16000000
8000	64000000
16000	256000000

Counting Instructions (4 of 5)

```
>"""
File: countfib.py
Prints the number of calls of a recursive Fibonacci
function with problem sizes that double.
"""

from counter import Counter

def fib(n, counter):
    """Count the number of calls of the Fibonacci function."""
    counter.increment()
    if n < 3:
        return 1
    else:
        return fib(n - 1, counter) + fib(n - 2, counter)

problemSize = 2
print("%12s%15s" % ("Problem Size", "Calls"))
for count in range(5):
    counter = Counter()
    # The start of the algorithm
    fib(problemSize, counter)
    # The end of the algorithm
    print("%12d%15s" % (problemSize, counter))
    problemSize *= 2
```

Counting Instructions (5 of 5)

Figure 3-4: The output of a tester program that runs the Fibonacci function

Problem Size	Calls
2	1
4	5
8	41
16	1973
32	4356617

Measuring the Memory Used by an Algorithm

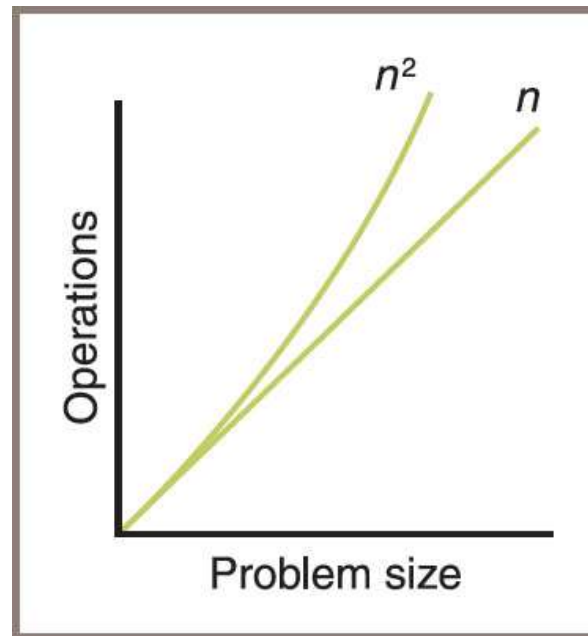
- A complete analysis of the resources used by an algorithm includes the amount of memory required:
 - Once again, focus on rates of potential growth
- Some algorithms require the same amount of memory to solve any problem
- Other algorithms require more memory as the problem size gets larger
- Later chapters consider several of these algorithms

Complexity Analysis

- This section focuses on developing a method of determining the efficiency of algorithms that allows you to rate them
 - Independently of platform-dependent timings or impractical instruction counts
- This method is called complexity analysis:
 - Entails reading the algorithm and using pencil and paper to work out some simple algebra

Orders of Complexity (1 of 5)

Figure 3-5: A graph of the amounts of work done in the tester programs



Orders of Complexity (2 of 5)

Table 3.1 The Amounts of Work in the Tester Programs

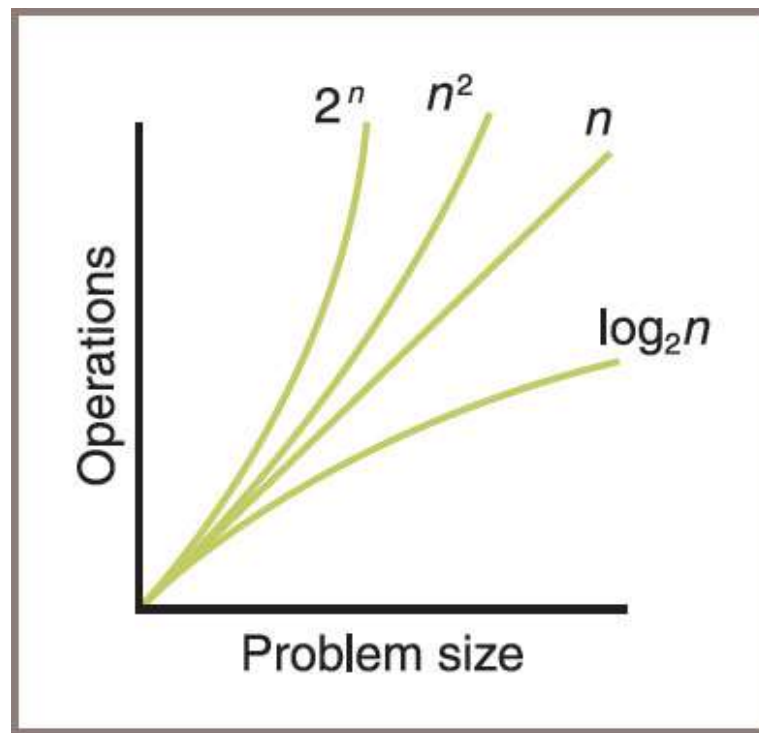
Problem Size	Work of the First Algorithm	Work of the Second Algorithm
2	2	4
10	10	100
1000	1000	1,000,000

Orders of Complexity (3 of 5)

- Performances of these algorithms differ by an order of complexity:
 - First algorithm is linear – work grows in direct proportion to the size of the problem
 - Second algorithm is quadratic – work grows as a function of the square of the problem size
- Other orders:
 - Constant – requires the same number of operations for any problem size
 - Logarithmic – amount of work is proportional to the $\log_2$ of the problem size
 - Polynomial time – grows at a rate of n^k , where k is a constant greater than 1
 - Exponential – example rate of growth of this order is 2^n

Orders of Complexity (4 of 5)

Figure 3-6: A graph of some sample orders of complexity



Orders of Complexity (5 of 5)

Table 3.2 Some Sample Orders of Complexity

n	Logarithmic ($\log_2 n$)	Linear (n)	Quadratic(n^2)	Exponential (2^n)
100	7	100	10,000	Off the chart
1,000	10	1000	1,000,000	Off the chart
1,000,000	20	1,000,000	1,000,000,000,000	Really off the chart

Big-O-Notation (1 of 2)

- An algorithm rarely performs a number of operations exactly equal to n , n^2 , or K^n :
 - An algorithm usually performs other work in the body of a loop, above the loop, and below the loop
- Whenever the amount of work is expressed as a polynomial, one term is dominant:
 - As n becomes large, the dominant term becomes so large that you can ignore the amount of work represented by the other terms
- For example,
 - In the polynomial $\frac{1}{2}n^2 - \frac{1}{2}n$, you focus on the quadratic term, $\frac{1}{2}n^2$, in effect dropping the linear term, $\frac{1}{2}n$, from consideration
 - You can also drop the coefficient $\frac{1}{2}$ because the ratio between $\frac{1}{2}n^2$ and n^2 does not change as n grows

Big-O-Notation (2 of 2)

- One notation used to express the efficiency or computational complexity of an algorithm is called *big-O notation*:
 - “O” stands for “on the order of,” a reference to the order of complexity of the work of the algorithm
- For example,
 - The order of complexity of a linear-time algorithm is $O(n)$
 - Big-O notation formalizes our discussion of orders of complexity

The Role of the Constant of Proportionality

- Constant of proportionality
 - Involves the terms and coefficients that are usually ignored during the big-O analysis
- For example,
 - The work performed by a linear time algorithm might be expressed as $work = 2 * size$, where the constant of proportionality, 2 in this case, is $work / size$
- Try to determine the constant of proportionality for the first algorithm discussed in this chapter – the code:

```
work = 1
for x in range(problemSize):
    work += 1
    work -= 1</pre>
```


Search Algorithms

- This section covers several algorithms that can be used for searching and sorting lists
- You will
 - Learn the design of an algorithm
 - See its implementation as a Python function
 - See an analysis of the algorithm's computational complexity

Search for the Minimum

- Python code for algorithm in function `indexOfMin`:

```
def indexOfMin(lyst):  
    """Returns the index of the minimum item."""  
    minIndex = 0  
    currentIndex = 1  
    while currentIndex < len(lyst):  
        if lyst[currentIndex] < lyst[minIndex]:  
            minIndex = currentIndex  
        currentIndex += 1  
    return minIndex
```

Sequential Search of a List

- Python code for a sequential search function:

```
def sequentialSearch(target, lyst):  
    """Returns the position of the target item if found,  
    or -1 otherwise."""  
    position = 0  
    while position < len(lyst):  
        if target == lyst[position]:  
            return position  
        position += 1  
    return -1
```

Best-Case, Worst-Case, and Average-Case Performance

- The performance of some algorithms depends on the placement of the data processed:
 - The sequential search algorithm does less work to find a target at the beginning of a list than at the end of the list
- Analysis of a sequential search considers three cases:
 - In the worst case, the target item is at the end of the list or not in the list at all:
 - Algorithm must visit every item and perform n iterations for a list of size n
 - Thus, the worst-case complexity of a sequential search is $O(n)$
 - In the best case, the algorithm finds the target at the first position, after making one iteration, for an $O(1)$ complexity
 - To determine the average case, add the number of iterations required to find the target at each possible position and divide the sum by n :
 - Thus, the algorithm performs $\frac{(n + n - 1 + n - 2 + \dots + 1)}{n}$, or $\frac{(n + 1)}{2}$ iterations

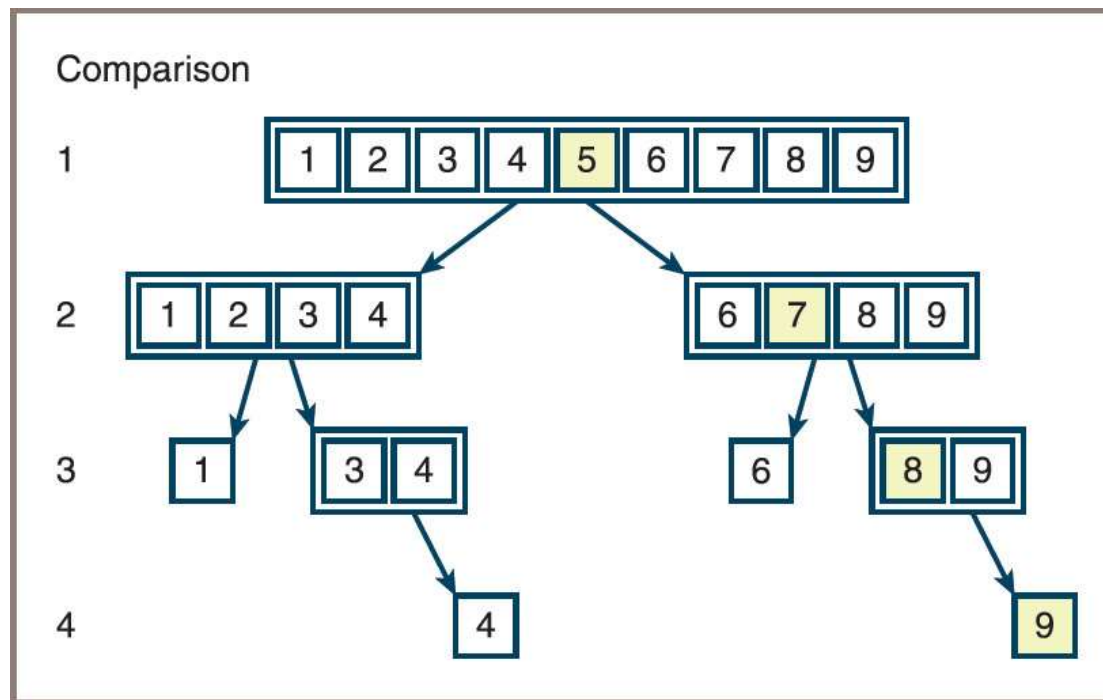
Binary Search of a Sorted List (1 of 2)

- Python code for the binary search function:

```
def binarySearch(target, sortedLyst):  
    left = 0  
    right = len(sortedLyst) - 1  
    while left <= right:  
        midpoint = (left + right) // 2  
        if target == sortedLyst[midpoint]:  
            return midpoint  
        elif target < sortedLyst[midpoint]:  
            right = midpoint - 1  
        else:  
            left = midpoint + 1  
    return -1
```

Binary Search of a Sorted List (2 of 2)

Figure 3-7: The items of a list visited during a binary search for 10



Comparing Data Items (1 of 3)

- To allow algorithms to use the comparison operators `==`, `<`, and `>` with a new class of objects,
 - The programmer should define the `__eq__`, `__lt__`, and `__gt__` methods in that class
- If you do this, the methods for the other comparison operators will automatically be provided
- The header of `__lt__` is the following:
 - `def __lt__(self, other):`
- This method returns **True** if **self** is less than **other**, or **False** otherwise

Comparing Data Items (2 of 3)

- The criteria for comparing the objects depend on their internal structure and on the manner in which they should be ordered
- For example:

```
class SavingsAccount(object):  
    """This class represents a savings account  
    with the owner's name, PIN, and balance."""  
  
    def __init__(self, name, pin, balance = 0.0):  
        self.name = name  
        self.pin = pin  
        self.balance = balance  
  
    def __lt__(self, other):  
        return self.name < other.name  
  
    # Other methods, including __eq__
```


Comparing Data Items (3 of 3)

```
<CDTX>>> s1 = SavingsAccount("Ken", "1000", 0)
>>> s2 = SavingsAccount("Bill", "1001", 30)
>>> s1 < s2
False
>>> s2 < s1
True
>>> s1 > s2
True
>>> s2 > s1
False
>>> s2 == s1
False
>>> s3 = SavingsAccount("Ken", "1000", 0)
>>> s1 == s3
True
>>> s4 = s1
>>> s4 == s1
True
```

Basic Sort Algorithms

- Each of the Python sort functions that are developed operates on a list of integers and uses a **swap** function to exchange the positions of two items in the list
- Here is the code for that function:

```
def swap(lyst, i, j):  
    """Exchanges the items at positions i and j."""  
    # You could say lyst[i], lyst[j] = lyst[j], lyst[i]  
    # but the following code shows what is really going on  
    temp = lyst[i]  
    lyst[i] = lyst[j]  
    lyst[j] = temp
```

Selection Sort (1 of 2)

- Figure 3.8 shows the states of a list of five items after each search and swap pass of a selection sort:
 - The two items just swapped on each pass have asterisks next to them and the sorted portion of the list is shaded

Figure 3-8: A trace of data during a selection sort

UNSORTED LIST	AFTER 1st PASS	AFTER 2nd PASS	AFTER 3rd PASS	AFTER 4th PASS
5	1*	1	1	1
3	3	2*	2	2
1	5*	5	3*	3
2	2	3*	5*	4*
4	4	4	4	5*

Selection Sort (2 of 2)

- Python function for a selection sort:

```
def selectionSort(lyst):  
    i = 0  
    while i < len(lyst) - 1:      # Do n - 1 searches  
        minIndex = i           # for the smallest  
        j = i + 1  
        while j < len(lyst):    # Start a search  
            if lyst[j] < lyst[minIndex]:  
                minIndex = j  
            j += 1  
        if minIndex != i:      # Exchange if needed  
            swap(lyst, minIndex, i)  
        i += 1
```

Bubble Sort (1 of 2)

- Figure 3.9 shows a trace of the bubbling process through a list of five items:
 - This process makes four passes through a nested loop to bubble the largest item down to the end of the list

Figure 3-9: A trace of data during a bubble sort

UNSORTED LIST	AFTER 1st PASS	AFTER 2nd PASS	AFTER 3rd PASS	AFTER 4th PASS
5	4*	4	4	4
4	5*	2*	2	2
2	2	5*	1*	1
1	1	1	5*	3*
3	3	3	3	5*

Bubble Sort (2 of 2)

- You can make a minor adjustment to the bubble sort to improve its best-case performance to linear
- Modified bubble sort function:

```
def bubbleSortWithTweak(lyst):  
    n = len(lyst)  
    while n > 1:  
        swapped = False  
        i = 1  
        while i < n:  
            if lyst[i] < lyst[i - 1]: # Exchange if needed  
                swap(lyst, i, i - 1)  
                swapped = True  
            i += 1  
        if not swapped: return # Return if no swaps  
        n -= 1
```

Insertion Sort (1 of 3)

- An insertion sort attempts to exploit the partial ordering of the list in a different way
- The strategy:
 - On the i th pass through the list, where i ranges from 1 to $n - 1$, the i th item should be inserted into its proper place among the first i items in the list
 - After the i th pass, the first i items should be in sorted order:
 - This process is analogous to the way in which many people organize playing cards in their hands. That is, if you hold the first $i - 1$ cards in order, you pick the i th card and compare it to these cards until its proper spot is found
 - Insertion sort consists of two loops:
 - The outer loop traverses the positions from 1 to $n - 1$
 - For each position i in this loop, you save the item and start the inner loop at position $i - 1$
 - For each position j in this loop, you move the item to position $j + 1$ until you find the insertion point for the saved (i th) item

Insertion Sort (2 of 3)

- Code for the `insertionSort` function:

```
def insertionSort(lyst):  
    i = 1  
    while i < len(lyst):  
        itemToInsert = lyst[i]  
        j = i - 1  
        while j >= 0:  
            if itemToInsert < lyst[j]:  
                lyst[j + 1] = lyst[j]  
                j -= 1  
            else:  
                break  
        lyst[j + 1] = itemToInsert  
        i += 1
```


Insertion Sort (3 of 3)

Figure 3-10: A trace of data during insertion sort

UNSORTED LIST	AFTER 1st PASS	AFTER 2nd PASS	AFTER 3rd PASS	AFTER 4th PASS
2	2	1*	1	1
5 ←	5 (no insertion)	2	2	2
1	1 ←	5	4*	3*
4	4	4 ←	5	4
3	3	3	3 ←	5

Best-Case, Worst-Case, and Average-Case Performance Revisited

- Best case
 - Under what circumstances does an algorithm do the least amount of work?
 - What is the algorithm's complexity in this best case?
- Worst case
 - Under what circumstances does an algorithm do the most amount of work?
 - What is the algorithm's complexity in this worst case?
- Average case
 - Under what circumstances does an algorithm do a typical amount of work?
 - What is the algorithm's complexity in this typical case?

Faster Sorting (1 of 2)

- The secret to these better algorithms is a divide-and-conquer strategy:
 - Each algorithm finds a way of breaking the list into smaller sublists
 - These sublists are then sorted recursively
- Ideally, if the number of these subdivisions is $\log(n)$ and the amount of work needed to rearrange the data on each subdivision is n ,
 - Then the total complexity of such a sort algorithm is $O(n \log n)$

Faster Sorting (2 of 2)

- Table 3.3 Comparing $n \log n$ and n^2

n	$n \log n$	n^2
512	4608	262,144
1024	10240	1,048,576
2048	22,458	4,194,304
8192	106,496	67,108,864
16,384	229,376	268,435,456
32,768	491,520	1,073,741,824

Overview of Quicksort (1 of 5)

- Outline of the strategy used in the quicksort algorithm:
 - Select the item at the list's midpoint (called the pivot)
 - Partition items in the list so that all items less than the pivot are moved to the left of the pivot (rest are moved to the right)
 - Divide and conquer
 - Process terminates each time it encounters a sublist with fewer than two items
- Partitioning
 - The most complicated part of the algorithm is partitioning the items in a sublist
 - See Figures 3.11 and 3.12 on the following slides for an illustration of the steps

Overview of Quicksort (2 of 5)

Figure 3-1 1: Partitioning a sublist

Step	Action carried out	State of the list after the action
	Let the sublist consist of the numbers shown with a pivot of 14.	12 19 17 18 14 11 15 13 16
1	Swap the pivot with the last item.	12 19 17 18 16 11 15 13 14
2	Establish the boundary before the first item.	: 12 19 17 18 16 11 15 13 14
3	Scan for the first item less than the pivot.	: 12 19 17 18 16 11 15 13 14
4	Swap this item with the first item after the boundary. In this example, the item gets swapped with itself.	: 12 19 17 18 16 11 15 13 14
5	Advance the boundary.	12 : 19 17 18 16 11 15 13 14
6	Scan for the next item less than the pivot.	12 : 19 17 18 16 11 15 13 14
7	Swap this item with the first item after the boundary.	12 : 11 17 18 16 19 15 13 14
8	Advance the boundary.	12 11 : 17 18 16 19 15 13 14
9	Scan for the next item less than the pivot.	12 11 : 17 18 16 19 15 13 14
10	Swap this item with the first item after the boundary.	12 11 : 13 18 16 19 15 17 14
11	Advance the boundary.	12 11 13 : 18 16 19 15 17 14
12	Scan for the next item less than the pivot; however, there is not one.	12 11 13 : 18 16 19 15 17 14
13	Interchange the pivot with the first item after the boundary. At this point, all items less than the pivot are to the pivot's left and the rest are to its right.	12 11 13 : 14 16 19 15 17 18

Overview of Quicksort (3 of 5)

Figure 3-12: Partitioning a sublist

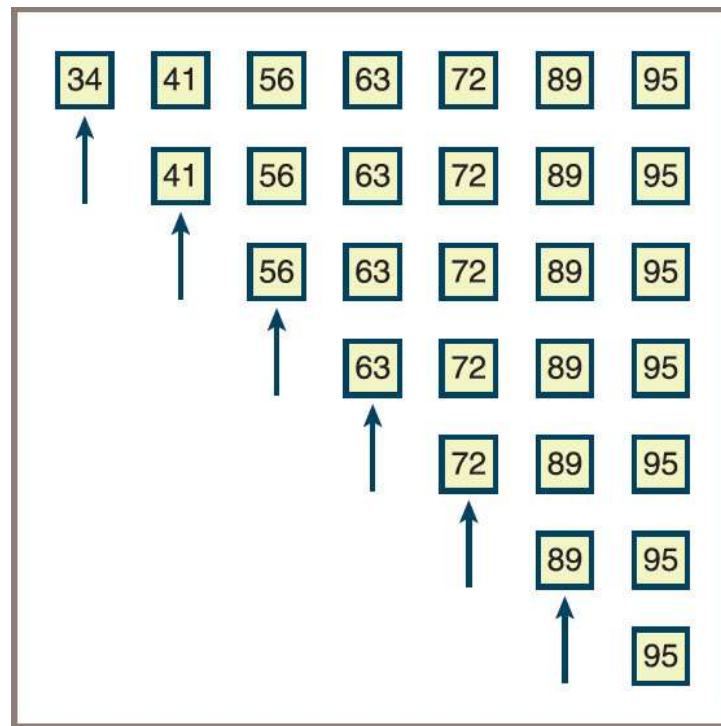
List Segment	Pivot Item
12 19 17 18 14 11 15 13 16	14
12 11 13	11
13 12	13
12	
16 19 15 17 18	15
19 18 17 16	18
16 17	16
17	
19	

Overview of Quicksort (4 of 5)

- Complexity analysis of quicksort
 - During the first partition operation, you scan all the items from the beginning of the list to its end
 - The amount of work during this operation is proportional to n , the list's length
 - The amount of work after this partition is proportional to the left sublist's length plus the right sublist's length, which together yield $n - 1$
 - When these sublists are divided, there are four pieces whose combined length is approximately n , so the combined work is proportional to n yet again
 - As the list is divided into more pieces, the total work remains proportional to n .
 - To complete the analysis, you need to determine how many times the lists are partitioned

Overview of Quicksort (5 of 5)

Figure 3-13: A worst-case scenario for quicksort (arrows indicate pivot elements)



Implementation of Quicksort (1 of 2)

- The quicksort algorithm is most easily coded using a recursive approach
- The following script defines a top-level quicksort function for the client, a recursive quicksortHelper function to hide the extra arguments for the endpoints of a sublist, and a partition function:
 - Script is continued on the next slide

```
def quicksort(lyst):  
    quicksortHelper(lyst, 0, len(lyst) - 1)  
  
def quicksortHelper(lyst, left, right):  
    if left < right:  
        pivotLocation = partition(lyst, left, right)  
        quicksortHelper(lyst, left, pivotLocation - 1)  
        quicksortHelper(lyst, pivotLocation + 1, right)
```

Implementation of Quicksort (2 of 2)

```
def partition(lyst, left, right):
    # Find the pivot and exchange it with the last item
    middle = (left + right) // 2
    pivot = lyst[middle]
    lyst[middle] = lyst[right]
    lyst[right] = pivot
    # Set boundary point to first position
    boundary = left
    # Move items less than pivot to the left
    for index in range(left, right):
        if lyst[index] < pivot:
            swap(lyst, index, boundary)
            boundary += 1
    # Exchange the pivot item and the boundary item
    swap(lyst, right, boundary)
    return boundary

# Earlier definition of the swap function goes here

import random

def main(size = 20, sort = quicksort):
    lyst = []
    for count in range(size):
        lyst.append(random.randint(1, size + 1))
    print(lyst)
    sort(lyst)
    print(lyst)

if __name__ == "__main__":
    main()
```

Merge Sort (1 of 7)

- A merge sort employs a recursive, divide-and-conquer strategy to break the $O(n^2)$ barrier:
 - Compute the middle position of a list and recursively sort its left and right sublists
 - Merge the two sorted sublists back into a single sorted list
 - Stop the process when sublists can no longer be subdivided
- Three Python functions collaborate:
 - Function called by users
 - Helper function that hides the extra parameters required by recursive calls
 - A function that implements the merging process

Merge Sort (2 of 7)

- Implementing the merging process
 - Merging process uses an array of the same size as the list
 - Array is called the **copyBuffer**
 - The buffer is allocated once in **mergeSort** and subsequently passed as an argument to **mergeSortHelper** and **merge**
- Code for **mergeSort**:

```
from arrays import Array
```

```
def mergeSort(lyst) :  
    # lyst list being sorted  
    # copyBuffer temporary space needed during merge  
    copyBuffer = Array(len(lyst))  
    mergeSortHelper(lyst, copyBuffer, 0, len(lyst) - 1)
```

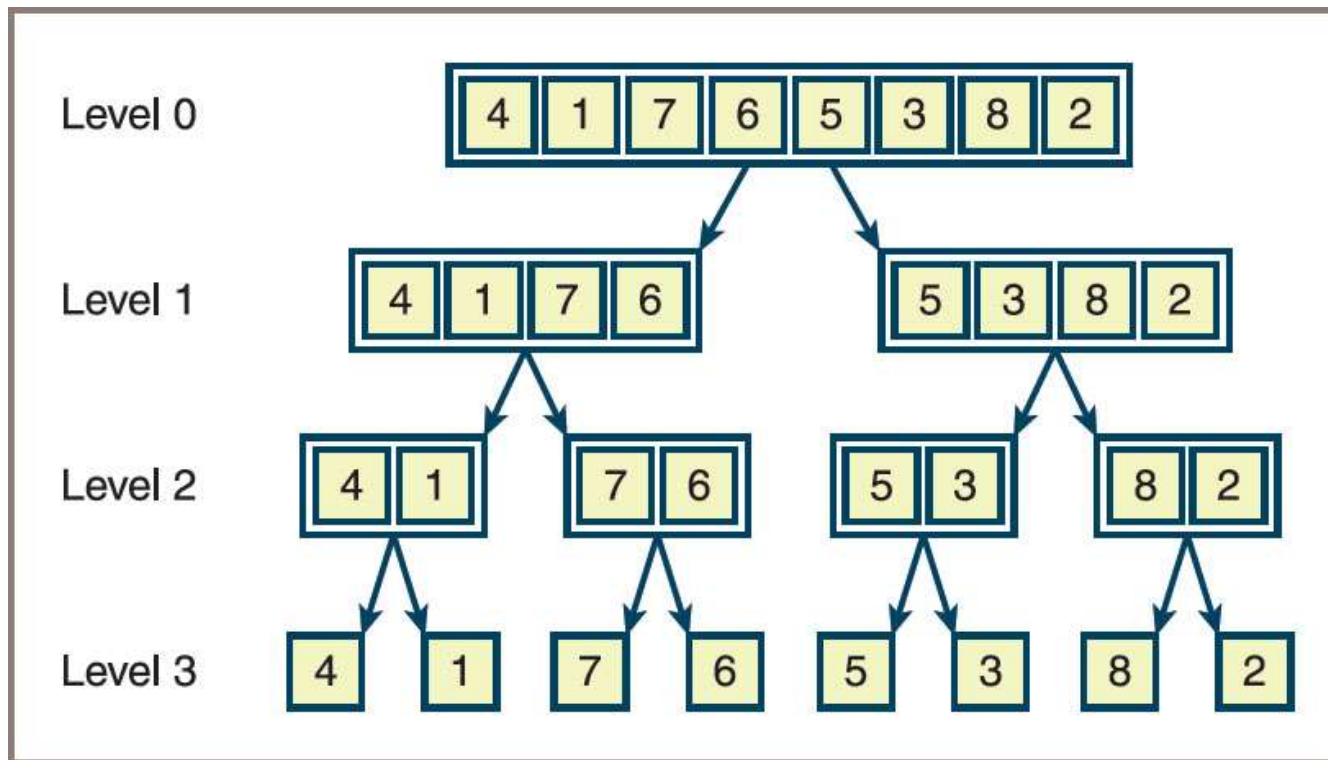
Merge Sort (3 of 7)

- Code for `mergeSortHelper`:

```
def mergeSortHelper(lyst, copyBuffer, low, high) :  
    # lyst list being sorted  
    # copyBuffer temp space needed during merge  
    # low, high bounds of sublist  
    # middle midpoint of sublist  
    if low < high:  
        middle = (low + high) // 2  
        mergeSortHelper(lyst, copyBuffer, low, middle)  
        mergeSortHelper(lyst, copyBuffer, middle + 1, high)  
        merge(lyst, copyBuffer, low, middle, high)
```

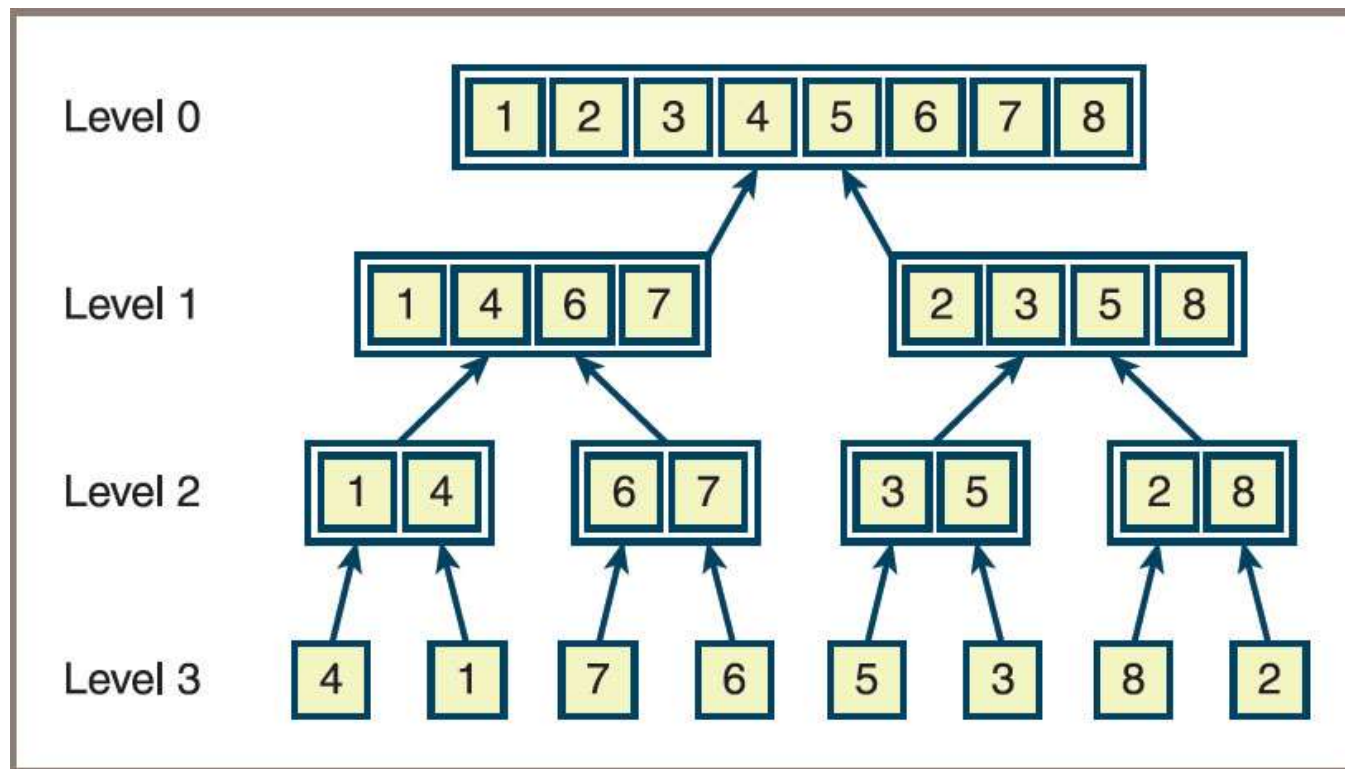
Merge Sort (4 of 7)

Figure 3-14: Sublists generated during calls of mergeSortHelper



Merge Sort (5 of 7)

Figure 3-15: Merging the sublists generated during a merge sort



Merge Sort (6 of 7)

- Code for the `merge` function:

```
def merge(lyst, copyBuffer, low, middle, high):
    # lyst      list that is being sorted
    # copyBuffer temp space needed during the merge process
    # low       beginning of first sorted sublist
    # middle    end of first sorted sublist
    # middle + 1 beginning of second sorted sublist
    # high      end of second sorted sublist
    # Initialize i1 and i2 to the first items in each sublist
    i1 = low
    i2 = middle + 1
    # Interleave items from the sublists into the
    # copyBuffer in such a way that order is maintained.
    for i in range(low, high + 1):
        if i1 > middle:
            copyBuffer[i] = lyst[i2] # First sublist exhausted
            i2 += 1
        elif i2 > high:
            copyBuffer[i] = lyst[i1] # Second sublist exhausted
            i1 += 1
        elif lyst[i1] < lyst[i2]:
            copyBuffer[i] = lyst[i1] # Item in first sublist <
            i1 += 1
        else:
            copyBuffer[i] = lyst[i2] # Item in second sublist <
            i2 += 1
    for i in range(low, high + 1): # Copy sorted items back to
        lyst[i] = copyBuffer[i] # proper position in lyst
```

Merge Sort (7 of 7)

- Complexity analysis for merge sort
 - The running time of the `merge` function is dominated by the two `for` statements, each of which loops $(\text{high} - \text{low} + 1)$ times
 - The function's running time is $O(\text{high} - \text{low})$
 - All the merges at a single level take $O(n)$ time
 - Because `mergeSortHelper` splits sublists as evenly as possible at each level, the number of levels is $O(\log n)$, and the maximum running time for this function is $O(n \log n)$ in all cases

Exponential Algorithm: Recursive Fibonacci (1 of 3)

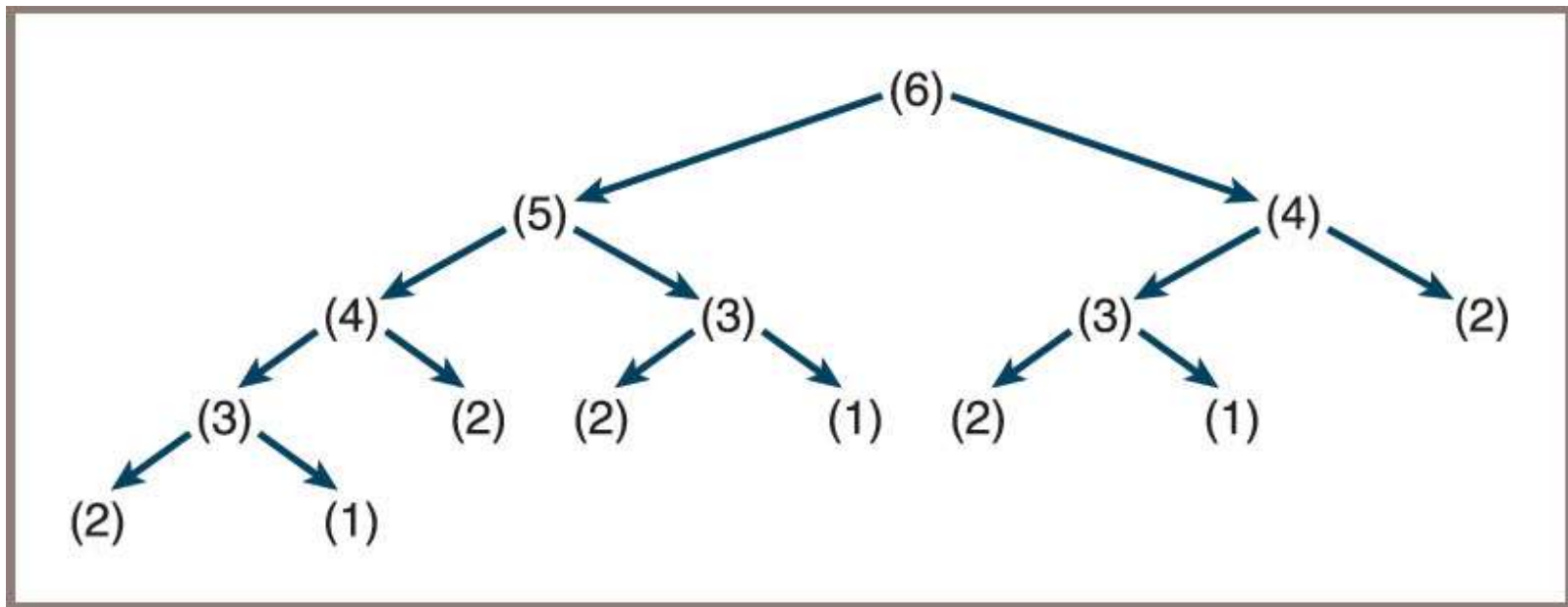
- Code for the Fibonacci function:

```
def fib(n) :  
    """The recursive Fibonacci function."""  
    if n < 3:  
        return 1  
    else:  
        return fib(n - 1) + fib(n - 2)
```

Exponential Algorithm: Recursive Fibonacci (2 of 3)

- Figure 3.16 shows the calls involved when using the recursive function to compute the sixth Fibonacci number

Figure 3-16: A call tree for fib(6)



Exponential Algorithm: Recursive Fibonacci (3 of 3)

- Exponential algorithms are generally impractical to run with anything but very small problem sizes
- Although recursive Fibonacci is elegant in its design,
 - There is a less beautiful but much faster version that uses a loop to run in linear time (see the next section)
- Alternatively, recursive functions that are called repeatedly with the same arguments can be made more efficient by a technique called *memorization*:
 - The program maintains a table of the values for each argument used with the function
 - Before the function recursively computes a value for a given argument, it checks the table to see if that argument already has a value
 - If so, that value is simply returned
 - If not, computation proceeds and the argument and value are added to the table afterward

Converting Fibonacci to a Linear Algorithm (1 of 2)

- An alternate algorithm starts a loop if n is at least the third Fibonacci number:
 - This number will be at least the sum of the first two ($1 + 1 = 2$)
- The loop computes this sum and then performs two replacements:
 - The first number becomes the second one, and the second one becomes the sum just computed
- The loop counts from 3 through n
- The sum at the end of the loop is the n th Fibonacci number

Converting Fibonacci to a Linear Algorithm (2 of 2)

- Pseudocode for this algorithm:

Set sum to 1

Set first to 1

Set second to 1

Set count to 3

While count <= N

 Set sum to first + second

 Set first to second

 Set second to sum

 Increment count

Chapter Summary (1 of 2)

- Different algorithms for solving the same problem can be ranked according to the time and memory resources that they require
- You can measure the running time of an algorithm empirically with the computer's clock
- Counting instructions provide another empirical measurement of the amount of work that an algorithm does
- The rate of growth of an algorithm's work can be expressed as a function of the size of its problem instances
- Big-O notation is a common way of expressing an algorithm's run-time behavior
- Common expressions of run-time behavior are $O(\log_2 n)$ (logarithmic), $O(n)$ (linear), $O(n^2)$ (quadratic), and $O(k^n)$ (exponential)

Chapter Summary (2 of 2)

- An algorithm can have different best-case, worst-case, and average-case behaviors
- In general, it is better to try to reduce the order of an algorithm's complexity than it is to try to enhance performance by tweaking the code
- A binary search is substantially faster than a sequential search
- The $n \log n$ sort algorithms use a recursive, divide-and-conquer strategy to break the n^2 barrier
- Exponential algorithms are primarily of theoretical interest and are impractical to run with large problem sizes